NEAR EAST UNIVERSITY

FACULTY OF ENGINEERING

DEPARTMENT OF SOFTWARE ENGINEERING

SRS DOCUMENT -304

Prepared By: IFEDIATU MARY CHISOANYI

Student ID: 20226411

1. Introduction

Software Requirements Analysis is a critical phase of software engineering that focuses on understanding stakeholder needs, defining system requirements, and ensuring that software solutions address real-world problems effectively. The success of a software project largely depends on the quality and completeness of its requirements.

This report presents the laboratory activities completed during the SWE304 Software Requirements Analysis course. The laboratories covered various requirements engineering techniques including domain engineering, requirement problem identification, system understanding, problem modelling, requirements elicitation, stakeholder workshops, vision documentation, product scope management, quality process models, and software requirements analysis tools.

Each laboratory exercise focused on applying theoretical concepts to practical case studies such as Smart Laboratory Access Systems, Campus Bicycle Rental Systems, and Supermarket Automation Systems. Through these activities, important software engineering concepts such as stakeholder analysis, requirement specification, scope management, quality assurance, and requirements traceability were explored.

The purpose of this report is to consolidate the outcomes of all laboratory exercises into a single document and demonstrate the application of requirements engineering principles throughout the software development lifecycle.

1.2 Course Laboratory Overview

The SWE304 Software Requirements Analysis course provided practical exposure to the principles, techniques, and tools used in software requirements engineering. Through a series of laboratory exercises, students were introduced to the processes involved in understanding problems, identifying stakeholder needs, defining requirements, managing project scope, and ensuring software quality.

Each laboratory exercise focused on a specific aspect of requirements engineering and software analysis. Different case studies were used throughout the semester to demonstrate how these concepts can be applied in real-world situations.

The laboratory activities completed during the semester are summarized below:

Lab 1: Domain Engineering

This laboratory introduced the concept of domain engineering and the identification of common and variable features within a software domain. The exercise focused on reusable assets, commonality analysis, and variability analysis as a foundation for software product line engineering.

Lab 2: Requirement Problems

This laboratory focused on identifying problems in poorly written requirements. Ambiguous, incomplete, inconsistent, and unverifiable requirements were analyzed and improved using accepted requirements engineering principles.

Lab 3: System Understanding

This laboratory emphasized understanding a problem domain from a stakeholder perspective. Activities included stakeholder identification, stakeholder analysis, system boundaries, assumptions, constraints, and understanding the operational environment of a software system.

Lab 4: Problem Modelling

This laboratory introduced modelling techniques used to represent software problems visually. Context diagrams, system boundaries, external entities, and problem analysis models were developed to improve communication between stakeholders and developers.

Lab 5: Requirements Elicitation

This laboratory focused on gathering requirements from stakeholders using elicitation techniques. Missing information, stakeholder needs, functional requirements, and non-functional requirements were identified and documented.

Lab 6: Requirements Elicitation Workshop

This laboratory extended the elicitation process through stakeholder interviews and workshops. Feedback was collected, analyzed, and transformed into system requirements while considering technical, organizational, and financial constraints.

Lab 7: Vision Document and Storyboarding

This laboratory focused on defining the overall vision of a proposed software system. High-level goals, objectives, target users, system features, and storyboard representations were created to communicate the intended solution effectively.

Lab 8: Product Scope and Change Management

This laboratory examined project scope definition and the challenges associated with scope creep. Change requests were analyzed to determine their impact on project cost, schedule, complexity, and overall feasibility.

Lab 9: Quality Process Model

This laboratory explored software quality management and software development process models. Different development approaches were evaluated, and project roles, activities, tasks, and deliverables were identified.

Lab 10: Software Requirements Analysis Tool Case Study (Jira)

This laboratory involved the evaluation of Jira as a requirements management tool. Its capabilities, limitations, traceability features, reporting mechanisms, and suitability for software requirements analysis were examined.

Collectively, these laboratory exercises provided practical experience in the complete requirements engineering lifecycle, from problem identification and requirements gathering to scope management, quality assurance, and requirements tracking.

TABLE OF CONTENTS

2. DOMAIN ENGINEERING:

2.1 Domain engineering is the process of analyzing a specific software domain to identify common and variable features that can be reused across multiple systems. It provides a foundation for software product line engineering by promoting reuse, reducing development effort, and improving consistency among related software products. This laboratory focused on identifying domain characteristics, performing commonality and variability analysis, and examining reusable assets within an enterprise systems domain.

2.2 ENTERPRİSE SYSTEMS.

This domain covers large organısatıon wide software that supports core business operations across departments such as finance, HR, supply chain, CRM etc. Enterprıse system are built to manage complex operations across the entire organısatıon. This domain includes systems like the SAP ERP systems, CRM PLATFORMS AND HR systems.

2.3 COMMONALITY AND VARIABILITY ANALYSIS

AREA	COMMONALITY	VARIABILITY

USERS	User and role management	Role complexity and hierarchy
DATA MANAGEMENT	Persistent storage, transactions	Data models per business module
WORKFLOW	Process execution capability	Approval chains and escalation rules
INTEGRATION	API or messaging capability	External systems integrated
DEPLOYMENT	Enterprise-grade hosting	Cloud vs on-premise vs hybrid
COMPLIANCE	Audit logging	Industry specific regulations
SCALABILTY	Multi-user support	High availability and distributed scaling

identifying common features enables reuseable foundations and identifying variable features enables controlled customization and together they turn enterprise development from repeated construction into structured product line engineering.

2.4 REUSEABLE ASSETS IN DOMAIN ENGINEERING

Domain Engineering is the systematic process of analyzing a specific area to identify commonalities and variabilities among similar systems with the goal of creating reuseable assets. WHILE Single-system development focuses on building one product from scratch based on a unique set of requirements.

Commonality and variability analysis is a main aspect of domain engineering and software product lines, it is essential for maximizing reuse and managing complexity by identifying shared features and points of variation across a system. Reuseable assets minimizes redundant work , reduces development time and costs by 20-30% and it streamlines application engineering by providing validated, configurable components.

3. REQUIREMENT PROBLEMS:

3.1 The quality of a software system is highly dependent on the quality of its requirements. Poorly written requirements can lead to misunderstandings, project delays, increased costs, and system failures. This laboratory focused on identifying common requirement problems such as ambiguity, incompleteness, inconsistency, and unverifiability. Different types of requirements were analyzed and improved to ensure clarity, correctness, and testability.

3.2 Solutions:

The system shall be very fast – Noise and Ambiguity. – This is vague and not measurable. Without specific performance metrics it cannot be tested or verified.

The application should use modern technology – Ambiguity – This does not specify any concrete tools or standards so it leaves too much room for interpretation

The software must be completely secure- Overspecification/implementation bias.- Security needs to be described in terms of specific controls or threat protections otherwise it cannot be proven or guaranteed.

Users can update records easily- Noise, silence and Ambiguity. – Different users have different skills. This requirement should define usuablity criteria such as number of steps , training time or success rate.

The system shall work in all environments- Wishful thinking, Unsatisfiablity. – “All Environments is unclear and likely impossible. It should specify the exact operating system, broswers, devices where the system must function.

- The system shall respond to user actions within 2 seconds under normal load.

The application shall be developed using python 3.5 and spring boot 3.

- The software shall require user authentication, use role-based access control and encrpy data using TLS 1.3
- Users shall be able to update a record in no more than 4 steps without assistance.
- The system shall run on Windows 11 and Ubuntu 21.09 and support the version of chrome and firefox.

3.3 Part2.

I. Business requirement – The organisation shall provide an online system for students to register for courses to reduce manual paperwork and save time.

II. User Requirements – Students shall be able to view available courses
   - Students shall be able to add or drop courses online.

III. Functional requirements- The system shall allow students to log in using a student ID and password
   - The system shall check for schedule conflicts before allowing course registration.

IV. Non- Functional requirements- The system shall respond within 5 seconds.
   - The system shall be available 99% of the time during registration.

3.4 Part 3.

I. Requirement problem happens when the system built is not what the user actually wants. This usually happens because the requirements were not clear, incomplete or misunderstood.

II. Sometimes stakeholders are not sure what they really need. Different people may also want different things. They may explain things in a vague way or change their minds later. That's why requirements can become unclear or conflict with each other.

III. Good requirements should be clear, specific and easy to understand. They should be measurable so they can be tested.

IV. Ambigity is dangerous because different people may understand the same requirewment in different ways. This can lead to mistakes, delays, extra costs and a system that does not meet user needs.

V. Developers use it to know what to build. Testers use it to check if the system works correctly. Project mangers use it to plan the project. Clients use it to make sure the system meets their needs. It works like an agreement for everyone involved.

4.SYSTEM UNDERSTANDING:

4.1 Before requirements can be specified, it is essential to understand the problem domain, stakeholders, system boundaries, and environmental constraints. System understanding helps analysts gain a comprehensive view of the problem that the software system is intended to solve. This laboratory focused on stakeholder identification, stakeholder analysis, system boundaries, and various constraints that influence software development and implementation.

The Scenario:

Near East University plans to introduce a Smart Laboratory Access System to control student entry into laboratories using ID cards or biometric authentication.

Currently:

- Unauthorized students enter laboratories.
- Lab attendance is recorded manually.
- Equipment misuse sometimes occurs.
- Lab supervisors struggle to monitor usage.
- The university wants a computerized solution.

4.2 Part 1 – Understanding the Problem

I. The university lacks a secure and automated method to control laboratory access and monitor usage which leads to unauthorized access and equipment misuse.

II. Lab attendance and usage are manually recorded which can lead which can lead to loss of information and also making it error prone.

b. Theres no system to identify student identity before entry

III. Solving only the visible problem like attendance recording and ignoring the underlying problem like Unauthorized access or equipment misuse may continue causing problems like security risks or administrative inefficiency or even resource loss.

IV. Near east university requires a smart laboratory access solution that ensures only authorized students can enter, keep precise attendance records, prevent improper use of lab resources, efficient monitoring usage and gives supervisors visibility into lab activities.

4.3 Part 2 – Stakeholder Identification

I. Student – uses the system to access labs, they are also directly affected by access policies
   - Lab instructors/Supervisors – they monitor attendance, lab usage and prevent unauthorized entry.
   - University Administrators – responsible for overseeing the system effectiveness.

II. Secondary:
   - Maintenance/ IT staff: ensures that biometric devices, ID card readers and lab equipments functions properly and also helps with troubleshooting, backups, or minor software issues.
   - External Auditors- Review lab security and usage records for compliance.

III. End Users – these are Students and lab staff who directly interact with the system for access and attendance.

IV. System administration – IT personnel responsible for configuring the system, managing user accounts, monitoring system performance and handling security settings.

4.4 Part 3 – System Boundary Analysis

I. Authentication module(ID card, biometric verification),

- Attendance tracking and logging
- Lab usage monitoring and reporting
- User account management
- Alerts for unauthorized access or misuse

II. Physical laboratory doors and locks

- Lab equipment itself(system monitors usage but does not directly control the equipment)

III. Students

- Lab Supervisors
- System Administration
- Maintanance staff/ Technicians
- External Auditors

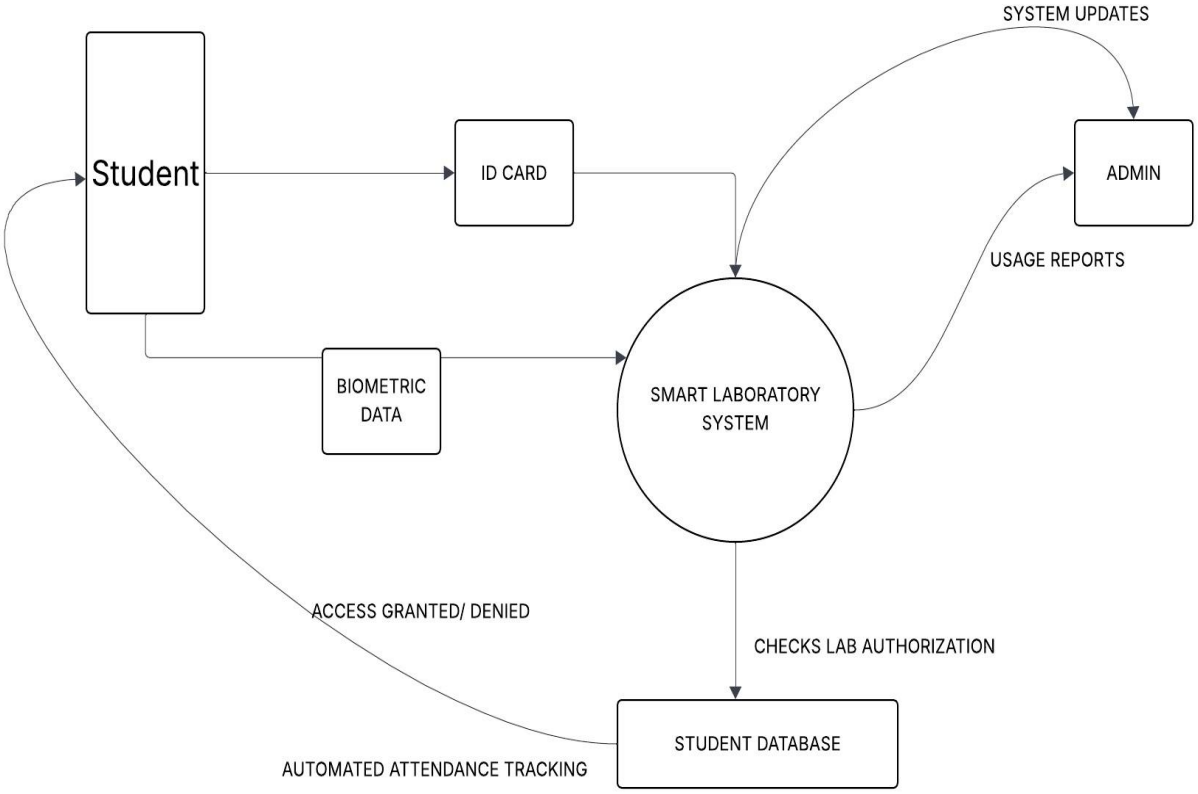
5. PROBLEM MODELLING:

5.1 Problem modelling is used to represent complex software problems through diagrams and structured models that improve communication and understanding among stakeholders. By visualizing system interactions and boundaries, analysts can identify requirements more effectively and reduce misunderstandings. This laboratory focused on developing context diagrams, analyzing external entities, and modelling the relationships between the system and its environment.

The Scenario:

Near East University plans to introduce a Smart Laboratory Access System to control student entry into laboratories using ID cards or biometric authentication.

5.2 Part 4 – Context Diagram Development



5.3 Part 5 – Constraints Identification

- **Hardware**

  Limited processing power of existing computers – performance constraint

  Insufficient storage capacity – capacity constraints

- **Network**

  Low bandwidth connections - performance constraint

  Intermittent network availability – Reliability constraint

- **Existing infrastructure**

  Legacy systems incompatible with new software – Compatibilty constraint

  Outdated database systems – Technical constraint

- **Lab working hours**

  Restricted access to labs outside scheduled times- Operational constraint

Limited time for equipment usage due to shared resources – Resource constraint

- **Staff availability**

  Key personnel on leave during critical phases - Resource constraint

  Limited availability of trained staff for testing - Operational constraint

- **Institutional policies**

  Mandatory approval processes for system changes – Procedural constraint

  Restrictions on software licensing types -policy constraint

- **Unauthorized access prevention**

  Requirement for multi-factor authentication – Security constraint

  Restriction on installing unverified software - Security constraint

- **Data protection**

  Encryption requirements for sensitive data - Security constraint

  Data retention limitations – Regulatory constraint

- **Authentication control**

  Password complexity and expiration policies - Security constraint

  Single sign-on (SSO) integration with existing systems- Technical constraint

Hardware

- Limited processing power can slow down software execution and restrict the complexity of features.
- Insufficient storage may require redesigning data handling or adding external storage solutions.

Network

- Low bandwidth can cause delays in data transmission, affecting real-time operations and user experience.
- Intermittent network availability can disrupt online services and require offline or caching solutions.

Existing infrastructure

- Legacy systems may force developers to use outdated technologies or limit integration with new systems.
- Outdated databases can restrict data operations and require migration strategies.

Lab working hours

- Restricted lab access can delay testing and prototyping, impacting project timelines.
- Limited equipment usage may require scheduling or prioritizing tests, slowing development cycles.

Staff availability

- Absences of key personnel can delay decision-making and critical development tasks.
- Limited trained staff for testing may lead to slower QA processes and potential errors.

Institutional policies

- Mandatory approval processes can lengthen development timelines and reduce flexibility.
- Licensing restrictions may limit software options or require extra compliance measures.

Unauthorized access prevention

- Multi-factor authentication increases security but may require additional coding and user training.
- Restrictions on installing unverified software can limit the use of potentially useful tools, requiring alternatives.

Data protection

- Encryption requirements can increase processing overhead and complexity in data handling.
- Data retention limits may force regular purging of records, affecting reporting and analysis.

Authentication control

- Password policies may require developers to implement additional logic for validation and reset flows.
- Integrating single sign-on can be complex and may introduce compatibility issues with existing systems.

5.4 Part 6

- System boundaries must be clearly defined before writing requirements because they establish ‘what is inside and outside the scope of the system’. This clarity helps ensure that the development team focuses on the right features, avoids unnecessary work, and prevents misunderstandings with stakeholders.
- Missing stakeholders can significantly affect project success because they represent perspectives, requirements, or approvals that the team might overlook. Essentially, each missing stakeholder increases the risk of gaps, misunderstandings, or conflicts, which can directly compromise project quality, schedule, and overall success.
- Ignoring constraints during requirement analysis can introduce several serious risks that affect the project’s cost, schedule, and quality. Risks examples includes scope creep, budget overruns, missed deadlines, security compliance etc.
- A context diagram helps bridge the gap between technical and non-technical stakeholders by providing a high-level visual representation of the system and how it interacts with external entities.
- Problem analysis helps prevent scope creep by focusing the project on the core issues that need to be solved before defining detailed requirements. By understanding the problem thoroughly, the team can distinguish between essential features and nice-to-have additions.

6.REQUIREMENT ELICITATION:

6.1 Requirements elicitation is the process of gathering information from stakeholders to determine their needs, expectations, and constraints. It is considered one of the most critical activities in requirements engineering because incomplete or inaccurate requirements can negatively affect the entire software development lifecycle. This laboratory focused on identifying missing information, gathering stakeholder needs, and documenting both functional and non-functional requirements for a proposed software solution.

The Scenerio

Near East University plans to introduce a Campus Bicycle Rental System for students. The system should allow students to rent bicycles around campus. Bicycles will be placed at different stations. The university administration provided the following description:

- Students should be able to rent bicycles.
- The system should track bicycle availability.
- Payment should be handled by the system.
- Bicycles should be returned to any station.
- The system should be secure and easy to use.

6.2 Part 1 – Identifying Missing Information

- The medium of security ie how to ensure system security
- The medium of payment – This is not specified clearly above
- How the tracking system will be implemented
- Map – The number of stations or location of stations not stated or directions missing
- Verification of rental after payment- how are you able to access the bicycle for rent after payment

6.3 Part 2 – Stakeholder Identification

2 . We need to understand the medium of security that is how the system should be secured before it can be implemented.

- The system should be specific about the medium of payment i.e visa card, debit card, Apple pay etc for better data and information security
- How to track the bicycle's availability needs to be specific whether its to install a real-time IOT technology or GPS tracking so that it can provide real-time data on availability. A map needs to be implemented on the system in other for users to know specific locations and stations where the bicycles are located.
- There should be a way to unlock or access the bicycle after payment for example QR codes, automatic keys etc to ensure absolute security.

6.4 Functional Requirement

- How should the student or user be able to access the system i.e Login to use the system
- What authentication method is suitable for the student /user use to access the bicyecle
- Who should be permitted to have access to the system
- Will there be a usage limit per student
- What should happen if a bicycle is returned late or damaged
- How many bicycle stations will be installed on campus and where will they be located?

6.5 Non- functional

- How quickly should the system respond when a student checks bicycle availability?
- How should the system handle failures such as network damages or station malfunction to ensure rentals are not lost or misrecorded
- Which platforms/browser should the system support(mobile phones, laptops, chrome, firefox etc)

6.6 Part 3 – Answer the following Questions:

Poorly elicited requirements can create several serious risks in a project:

- System failure to meet user needs: The final system may not solve the real problem students or administrators have.
- Project delays: Missing requirements often appear later in development, causing redesign and delays.
- Increased development cost: Fixing requirement mistakes after development has started is expensive.
- Low user satisfaction: Students and staff may avoid using the system if it does not work as expected.
- Operational problems**:** Issues such as incorrect bicycle tracking, payment errors, or security weaknesses could occur.

6.7 Interviews would likely be the most effective elsicitation technique for this system.

1. Through interviews, analysts can speak directly with stakeholders such as university administrators, IT staff, and possibly student representatives at Near East University. This allows detailed discussions where analysts can ask follow-up questions, clarify unclear points, and understand the university's specific needs for bicycle stations, payments, and security.
2. Interviews also help uncover hidden or implicit requirements that stakeholders might not think to mention in surveys or documents. Because the bicycle rental system involves operational rules, payments, and campus infrastructure, direct communication through interviews can provide deeper and more accurate information.

7. REQUIREMENT ELICITATION WORKSHOP

7.1 Requirements workshops provide an effective environment for stakeholders and analysts to collaboratively discuss system requirements, clarify expectations, and resolve uncertainties. Workshops often reveal hidden requirements, operational challenges, and organizational constraints that may not emerge through other elicitation techniques. This laboratory involved stakeholder interviews and workshop discussions aimed at evaluating an existing laboratory access system and proposing improvements through a centralized smart access management solution.

7.2 Stakeholder Interview Feedback on the Proposed Centralized IoT Smart Door Lock Management System for Near East University

Introduction

We conducted an interview with Professor Fadi to gather feedback on the current laboratory access system and to evaluate the proposed IoT-based solution. The goal was to understand the existing challenges, user needs, system requirements, and possible constraints.

Scenario Overview

We found that the university currently uses physical keys to control access to laboratories. This method leads to issues such as unauthorized access, lack of proper tracking, lost or duplicated keys, and inefficient manual management. As a result, a smarter access system is being considered.

Interview Objective

During the interview, we aimed to identify the main problems with the current system, determine who should access the labs, understand preferred authentication methods, and evaluate important factors such as security, cost, and institutional policies.

7.3 Summary of Stakeholder Feedback Current Problems

From the interview, we found that the key-based system is difficult to manage and not secure. Issues include lost or copied keys, lack of proper tracking, and no clear record of lab usage.

Key Finding: The current system lacks efficiency, accountability, and security**.**

User Needs

We learned that access should mainly be limited to authorized users such as instructors, depending on roles and schedules. Biometric authentication was preferred, with other options like QR codes also considered.

Key Finding: The system should support role-based access and secure authentication.

Required System Features

We identified that the system should include access logging, remote permission control, and centralized management of lab access.

Key Finding: Monitoring and remote management are essential.

Security and Reliability

We observed that security is a major concern due to valuable lab equipment. The system should detect unauthorized access and send alerts to relevant staff.

Key Finding: Strong security and real-time alerts are necessary.

7.4 Constraints and Considerations

We found that cost, infrastructure, and university policies are important factors. A fully automated system may be expensive, so a more practical solution like a smart key management system could be more suitable.

Key Finding: The system must be cost-effective and comply with institutional rules.

Overall Analysis

Based on the interview, we concluded that the current system is outdated and lacks proper control and monitoring. There is a clear need for a smarter and more secure solution.

7.5 Key Requirements Functional Requirements

From our findings, the system should:

- Allow access only to authorized users
- Support biometric and alternative authentication
- Record access logs (user, time, location)
- Allow remote permission management
- Send alerts for unauthorized access
- Enforce role-based and time-based access

7.6 Non-Functional Requirements

We identified the following important qualities:

- Security
- Reliability
- Fast performance
- Availability during lab hours
- Cost-effectiveness
- Scalability and maintainability

Recommended Implementation Approach We identified

two possible approaches:

Option 1: Full smart lock system

- Provides high security and automation
- More expensive and complex

Option 2: Smart key management system (recommended)

- Uses existing locks
- Lower cost and easier to implement
- Still improves security and tracking

7.7 Cost Breakdown

Option 1: Smart Lock System

Cost grows with every door.

- Cost per door (installed): $200 – $500
- 20 doors: $4,000 – $10,000

System setup:

- $1,000 – $5,000

Total Year 1:

$5,000 – $15,000

Ongoing:

- $1,200 – $4,800/year (subscriptions)

Option 2: Smart Key Management System

Cost does NOT depend on number of doors (mostly).

- Key cabinet system: $3,000 – $8,000
- Setup: $500 – $2,000

Total Year 1:

$3,500 – $10,000

Ongoing:

- $300 – $1,500/year

Why Key Management Is Cheaper

- No hardware per door
- No installation across multiple doors
- No per-door software fees
- Uses existing locks

Simple Rule

- Few doors (1–5): costs may look similar
- Many doors (10+): key management becomes significantly cheaper
- Large sites (20+ doors): smart locks can be 2–3× more expensive

7.8 Conclusion

In conclusion, we found that the current laboratory access system is inefficient and insecure. A centralized smart access system is needed to improve security, monitoring, and overall efficiency, while also considering cost and existing university policies.

8.VISION DOCUMENT AND STORYBOARDING

8.1 A vision document provides a high-level description of a proposed system by defining its purpose, goals, stakeholders, and expected benefits. Storyboarding complements the vision document by visually illustrating how users interact with the system to accomplish specific tasks. This laboratory focused on developing a vision for a campus bicycle rental system and creating storyboard representations of key user interactions.

8.2 Problem Statement:

Students face difficulty moving quickly across campus, especially during busy hours. Current transportation options are limited, causing delays and inconvenience. A more efficient and flexible solution is needed.

8.3 System Overview:

The system will allow students to rent bicycles from stations located around campus and return them to any station. It will track bicycle availability in real time and handle payments digitally.

Target Users:

- Students (primary users)
- University staff
- Administration and maintenance teams

.Goals and Objectives:

- Provide an easy and fast bicycle rental process
- Track bicycle availability in real time
- Ensure secure and simple payment methods
- Improve campus mobility
- Support environmentally friendly transportation

Features (High-Level)

- Rent bicycles using a mobile app or student ID
- View available bicycles at different stations
- Return bicycles to any station
- Integrated digital payment system
- Secure login and data protection

Constraints (Budget, Time, Legal)

- Budget: Limited funds may affect the number of bicycles and stations
- Time: The system must be developed within a fixed timeline
- Legal: Must comply with data protection and payment security regulations

Conclusion:
The system will provide a convenient, efficient, and sustainable way for students to travel across campus.

8.4 Story Board: Rent Bicycle Use case.

Use case: Rent Bicycle.

Story Board Type: Active Story board

Actors:

Student

Rent bicycle system

8.5Challenge Faced:

One challenge was keeping the storyboard simple while still showing all necessary steps, especially balancing detail with clarity so it's easy to understand without becoming too complex

Another was using icons alone can make some steps (like payment confirmation or system feedback) less clear without short text explanations.

8.6 Storyboard

TITLE

Storyboard: Rent a Bicycle Use case

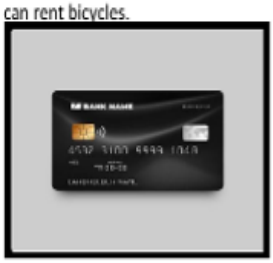
DATE 16/04/2026

PAGE 00

celtx

Student Login

The student accesses the system by logging in through the mobile app or scanning their student ID. This ensures that only authorized users can rent bicycles.



Confirm and pay

The student confirms the rental and completes the payment using a digital method. The system securely

View Available bicycles

After logging in, the student selects a station or views nearby stations. The system displays the number of available bicycles in real time, helping the student choose where to rent from

Scan the code

processes the payment and confirms the transaction

Select Bicycle

The student selects a specific bicycle from the available options. The system then shows details such as rental cost and usage terms.

Unlock Bicycle

The system unlocks the bicycle and starts the rental session.

9.PRODUCT SCOPE AND CHANGE MANAGENENT

9.1 Defining project scope is essential for ensuring that software development efforts remain focused on agreed objectives and deliverables. Effective change management helps organizations evaluate proposed modifications while minimizing the risks associated with scope creep. This laboratory focused on identifying project scope, evaluating change requests, and analyzing the impact of changes on project cost, schedule, and system complexity.

9.2 BICYCLE RENTAL SYSTEM

Define Project Scope: listing features

- Bicycle Availability Tracking
- Bicycle Rental and Return
- Integrated Payment System
- User Authentication
- Station Management

9.3 Part 2 – Identifying Scope Creep

All requests are likely to cause scope creep:

- Add real-time GPS tracking of bicycles
- Include mobile app support for both Android and iOS
- Add reward points for frequent users
- Integrate the system with external payment platforms

- Real-time GPS tracking → Scope Creep
  Adds new requirements like GPS hardware, live tracking, and map integration. Increases cost and system complexity beyond the original scope.
- Mobile apps (Android & iOS) → Scope Creep
  Requires building and maintaining two apps, adding extra development time, cost, and complexity compared to a basic system.
- Reward points system → Scope Creep
  Introduces a new feature (loyalty system) that needs user tracking, reward calculation, and management, which is outside core functionality.

- External payment integration → Potential Scope Creep
  Adds complexity through third-party APIs and security requirements. May not be scope creep if it was planned from the start.

9.4 Part 3 – Change Management Analysis

Real-Time GPS Tracking of Bicycles Problem

Analysis and Change Specification:

What is the requested change?

Add GPS tracking to each bicycle so the system can monitor their exact location in real time.

Why is it needed?

- To prevent theft or misuse
- To help locate lost or misplaced bicycles
- To improve operational control and redistribution of bikes across stations

Change Analysis and Costing

Impact on Time:

Development time will increase significantly due to hardware installation, backend integration, and real-time data testing.

Impact on Cost:

Costs will increase due to:

- GPS devices for each bicycle
- Data communication (SIM/data plans)
- Additional infrastructure for tracking and storage

Impact on System Complexity:

System complexity will increase due to:

- Real-time data processing
- Map integration
- Continuous device communication and maintenance

9.5 Problem Analysis and Change Specification

What is the requested change?

Develop mobile applications for both Android and iOS platforms.

Why is it needed?

- To improve user convenience and accessibility
- To provide a smoother user experience
- To enable features like notifications and quick bike access

Change Analysis and Costing

Impact on Time:

The project timeline will increase due to design, development, and testing of two applications (or a cross-platform solution).

Impact on Cost:

Costs will increase due to:

- Hiring mobile developers or using specialized tools
- Ongoing maintenance for both platforms
- App store deployment and updates

Impact on System Complexity:

System complexity will increase due to:

- Supporting multiple platforms
- Need for APIs to connect apps with backend
- Additional testing for performance and compatibility

10.QUALITY PROCESS MODEL

10.1 Software quality is achieved through structured development processes, clearly defined roles, and continuous verification activities throughout the software lifecycle. Selecting an appropriate software development model helps ensure that project objectives are achieved efficiently while maintaining quality standards. This laboratory focused on evaluating process models, identifying project roles and responsibilities, and defining activities and deliverables necessary for successful software development.

10.2 Problem statement:

The supermarket stocks a set of items . Customers take the items from different counters in required quantities and deposits them at their billing counter. The clerk enters the code number of items and their respective quantities/units. The managers of the supermarket plan to automate the existing system by implementing supermarket automation software (SAS).

The SAS should provide the following features

1. Print the bill containing serial numbers of the sales transaction, name each item, code number, quantity, unit price and total price at the end of the sales transaction. The bill should indicate the total amount payable
2. Print the sales statistics for items that are sold in the supermarket for any particular period. The sales statistics should indicate the quantity of an item sold, the price realized and the profit
3. Maintain the inventory of the various items of the supermarket. The manager upon query should be able to see the inventory details. In order to support the inventory management, the inventory of an item should be updated whenever an item is sold
4. Enable an employee to update the inventory when a new supply arrives

Questions:

1. Which SDLC model is best for dev the system and why
2. What are the roles, activities, tasks and deliverables for the design process.

10.3 SOLUTIONS:

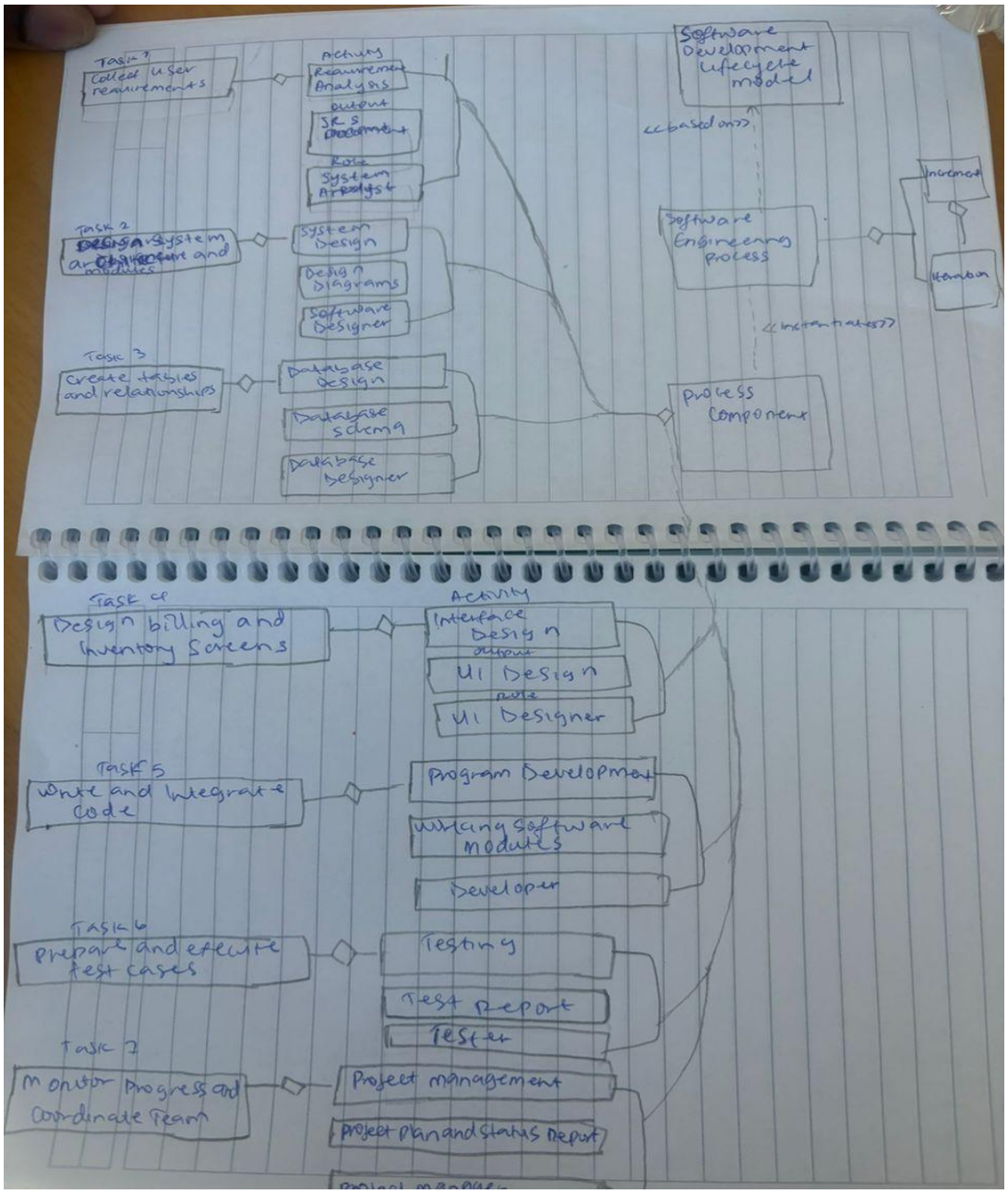
1. I think the best model for this is the ITERATIVE MODEL:

The system can be developed in small versions and improved continuously based on feedback from managers and employees. Features like billing, inventory, and sales reports can be refined in each iteration.

10.4

ROLE	Activity	Task	Deliverable
System Analyst	Requirement Analysis	Collect user requirement	SRS Document
Software Designer	System Design	Design system architecture and modules	Design Diagrams
UI Designer	Interface Design	Design billing and inventory screens	UI Design
Database Designer	Database Design	Create tables and relationships	Database schema
Developer	Program Development	Write and integrate code	Working Software modules
Tester	Testing	Prepare and execute tests cases	Test Report
Project Manager	Project Management	Monitor progress and coordinate teams	Project plan and status Reports

10.5 QUALITY PROCESS MODEL



11.SOFTWARE REQUIREMENT ANALYSIS TOOL – JIRA CASE STUDY

11.1 Modern software projects rely on specialized tools to manage requirements, track progress, support collaboration, and maintain traceability throughout the development lifecycle. Requirements management tools help organizations organize requirements, monitor changes, and improve communication among stakeholders. This laboratory examined Jira as a software requirements analysis and project management tool, evaluating its features, advantages, limitations, and practical applications in software development environments.

11.2 Software Requirement Analysis Tools

Choosen Tool – JIRA. Done

By:Ifediatu Mary Chiosanyi

11.3 Description Of Jira:

Jira is a leading project management and issue-tracking tool widely used for software requirement analysis, particularly in Agile teams to capture, prioritize, and track user stories and requirements from conception to release. While not a specialized document-based requirements tool, it excels at linking requirements to development tasks, fostering collaboration, and providing real-time visibility across the project lifecycle.

11.4 Key Aspects of Requirements Management in Jira:

- **Requirement Capture and Structure:** Users typically create custom issue types (e.g., "Requirement" or "User Story") to represent requirements, which can be organized in backlogs, epics, and sprints.
- **Jira Product Discovery:** A specialized tool within Jira designed for capturing, prioritizing, and visualizing product ideas and requirements before they enter the delivery phase.
- **Traceability and Linking:** Jira enables users to link requirements directly to related tasks, bugs, or epics, ensuring full traceability of, for instance, a feature from the initial requirement to the final test case.
- **Workflow Customization:** Requirements can be guided through specific workflows with custom fields (e.g., priority, source, risk) to ensure detailed documentation.
- **Confluence Integration:** Through native integration, requirements written in Confluence can be linked to Jira issues, providing comprehensive documentation that updates in real-time alongside development progress.
- **Reporting and Visibility:** Dashboards and JQL (Jira Query Language) allow teams to track the progress of requirements, ensuring all are covered and identifying potential bottlenecks.

11.5 1Limitations of Jira as an SRA Tool

Done by :Ivy Pauline

While Jira is a powerful tool for managing software development workflows, it presents several limitations when used specifically for Software Requirements Analysis (SRA). These limitations affect requirement clarity, traceability, and overall project management efficiency.

Lack of Structured Requirement Documentation

Jira organizes requirements as individual issues or tickets rather than as a cohesive document.

Requirements are fragmented across multiple tickets
Difficult to view the system as a complete specification Lacks a clear hierarchical structure

Impact:

This makes it challenging for stakeholders to understand the full scope of the system and reduces clarity in requirement communication.

Limited Traceability and Baselining

Jira does not provide strong built-in support for requirement traceability or version control.

Traceability between requirements, code, and testing is limited
No native traceability matrix
Weak support for baselining (snapshot of requirements at a given time)

Impact:

This can lead to difficulties in tracking requirement changes and ensuring consistency throughout the development lifecycle, especially in large or regulated projects.

Weak Resource and Project Management Capabilities

Jira lacks advanced project-level management features required for complex environments.

No built-in resource allocation tools
Limited capacity and workload management
No portfolio-level project tracking
No budget management functionality

Impact:

Teams may struggle to manage multiple projects efficiently, leading to resource conflicts, delays, and reduced productivity.

Limited Support for Decision-Making

Jira provides basic reporting but lacks advanced analytical tools.

No scenario simulation or forecasting features
Limited insights for strategic planning
Difficulty assessing the impact of changes

Impact:
This increases uncertainty in decision-making and makes it harder for managers to plan and optimize project outcomes.

11.6 Usability and Complexity Challenges

Jira can be difficult to use, especially for new users.

Complex interface
Requires training and onboarding
High administrative overhead for configuration

Additionally, excessive customization can:

Complicate workflows
Reduce system usability
Create inconsistency across teams

Impact:
These factors can reduce efficiency and increase the learning curve for teams adopting the tool.

Cost and Scalability Concerns

Jira can become expensive as project needs grow.

Pricing increases with users and advanced features
Many essential features require paid plugins Additional costs for integrations

Impact:
This may limit its accessibility for small teams or startups with budget constraints.

Limited Support for Non-Agile Methodologies

Jira is primarily designed for Agile development.

Less suitable for Waterfall or hybrid approaches
Requires significant customization for non-Agile workflows

Impact:
Organizations not following Agile methodologies may find Jira less effective and harder to adapt.

11.7 What We Learnt:

Done by :Triumph Aasam

One thing our team learnt from using this tool was how important proper requirement organization is in software development. Before using the tool, most of us only focused on writing requirements down, but we did not really think about how difficult it can become to manage them when changes start happening during development. The tool helped us see how requirements can be tracked, updated, and linked together instead of being scattered in different documents or chats.

We also learnt the importance of collaboration between team members and stakeholders. Since everyone could view updates and comments in one place, it reduced confusion and made communication clearer. It showed us that requirement analysis is not just about gathering information at the beginning of a project, but also about continuously managing changes as the project grows.

Another thing we noticed was how useful the tool was for keeping track of progress and avoiding misunderstandings. If a requirement was modified, it was easier to identify what changed and why. This helped us understand why companies use SRA tools in real-world projects where many people are working on the same system at the same time.

At the same time, we also learnt that tools alone do not solve requirement problems completely. The quality of the requirements still depends on how clearly the team communicates and documents information. Even with the tool, unclear requirements can still create confusion if they are not written properly.

11.8 Would you recommend Jira and Why?

Done by: Cindy Katua

Yes, but only for the right team

For medium to large software teams, Jira is hard to beat. It's built around how development works, that is, sprints, backlogs, epics, story points, releases. When your team is doing agile development, Jira doesn't fight you on terminology or workflow; it speaks your language natively.

The integrations are also a real competitive advantage. GitHub, Bitbucket, Confluence, Slack, CI/CD pipelines. Basically, it all connects without much friction. For a team that lives across multiple tools, that glue matters a lot in practice.

Reporting is another strong suit. Burndown charts, velocity tracking, sprint progress when a project manager or stakeholder asks, "where are we?", Jira can answer that question with data rather than vibes.

But at some point, Jira will frustrate you. Jira has a real complexity tax**.** Setting it up well takes time and thought. If you configure it badly of which many teams do, it becomes a bureaucratic nightmare where updating a ticket feels like filing a tax return.

For small teams or non-technical projects, it's genuinely overkill. A 3-person startup would be better served by Notion, Linear, or even Trello. Jira rewards teams that invest in it, but that investment is real.

The UI has also historically been clunky. Atlassian has improved this with the newer cloud version, but if you've used something like Linear, going back to Jira can feel like switching from a sports car to a delivery truck. The truck carries more, but, it's not a joy to drive.

Bottom line:

Jira is a professional-grade tool that rewards professional-grade usage. It's not the most lovable piece of software, and it'll humble you during setup, but for real-world software projects with real teams, real deadlines, and real stakeholders, it will always deliver. Just don't expect it to be fun. Expect it to be useful.

And that, for most projects, is exactly what you need.

12.Conclusion

This report presented the laboratory activities completed throughout the SWE304 Software Requirements Analysis course. The exercises covered key areas of requirements engineering including domain analysis, requirements identification, stakeholder analysis, problem modelling, elicitation techniques, vision development, scope management, quality assurance, and requirements management tools.

The laboratory activities demonstrated how software requirements evolve from an initial problem statement into a structured set of functional and non-functional requirements. They also highlighted the importance of stakeholder involvement, effective communication, change management, and quality processes in software development.

The case studies used throughout the semester provided practical experience in analyzing real-world software problems and applying appropriate requirements engineering techniques. Furthermore, the Jira case study illustrated the importance of modern tools in managing requirements, tracking changes, and supporting collaboration among project stakeholders.

Overall, the course provided valuable knowledge and practical skills that contribute to the successful planning, analysis, and development of software systems.

References

1. Sommerville, I. (2016). Software Engineering (10th Edition). Pearson Education.
2. Pressman, R. S., & Maxim, B. R. (2020). Software Engineering: A Practitioner's Approach (9th Edition). McGraw-Hill.
3. IEEE Computer Society. IEEE Recommended Practice for Software Requirements Specifications.
4. Atlassian. Jira Software Documentation.
5. SWE304 Software Requirements Analysis Laboratory Manual, Near East University.
6. Course lecture notes and presentation materials provided during SWE304.